KlinikKu: Mobile-Based Clinic Information System Using Flutter and Laravel REST API with MVC Architecture

1st Faiz Azzikri
Teknik Informatika
Universitas Muhammadiyah Bandung
Bandung, Indonesia
Faizzazzikri@gmail.com

2nd Mochamad firdausa arfiandi nur Pratama
Teknik Informatika
Universitas Muhammadiyah Bandung
Tasikmalaya, Indonesia
nevingaming001@gmail.com

3rd Muhammad Fadil Rizki
Teknik Informatika
Universitas Muhammadiyah Bandung
Lampung, Indonesia
fdlrzky8@gmail.com

4th Muhammad Rizqy Subagja
Teknik Informatika
Universitas Muhammadiyah Bandung
Sumedang, Indonesia
rizqysubagja99@gmail.com

Abstract—Clinical administrative services that are still performed manually often lead to various problems, such as delays in the patient registration process, data recording errors, disorganized queue management, and difficulties in storing medical records. This study aims to develop KlinikKu, a mobile-based clinical information system that implements a client–server architecture using Flutter as the client application and Laravel as the backend, which communicate via a REST API. The system is built using the Model–View–Controller (MVC) pattern, with Laravel Sanctum for authentication, Eloquent ORM for database access management, and MySQL as the data storage medium. Key features include the management of patient data, physician information, examination schedules, queues, and medical records, all integrated into a single system. The implementation of a client–server architecture allows for the separation of the mobile app and the backend, making system development and maintenance more structured. Through this integration, the system is expected to improve the efficiency of clinic administration, facilitate access to patient data and examination histories, and support faster, more accurate, and better-organized clinic services.

Keywords: Client–Server, Flutter, Laravel, Model–View–Controller (MVC), REST API, Clinic Information System.

I. INTRODUCTION

A. Background

Advances in information technology have driven digital transformation across various sectors, including healthcare. The use of information systems in

clinical services has become a solution to improve the effectiveness of data management, streamline administrative processes, and enhance the quality of patient care. Compared to manual processes, information systems can reduce the risk of recording errors, simplify data storage, and provide faster access to information for staff and medical personnel.

In practice, many clinics still face various administrative challenges, such as a time-consuming patient registration process, error-prone data recording, disorganized queue management, and difficulties in storing and retrieving patient medical histories. These conditions can affect the clinic's operational efficiency as well as the quality of care provided to patients. Therefore, an information system is needed that can integrate all administrative and clinical service processes into a single, centralized platform.

As mobile technology continues to evolve, the client–server architecture has become one of the most widely used approaches in modern application development. In this architecture, the client application serves as the user interface, while the server is responsible for handling business logic, data management, and communication with the database via REST API services. This approach allows for the separate development of the frontend and backend, thereby enhancing flexibility, scalability, and ease of system maintenance.

Based on these issues, this study developed KlinikKu, a mobile-based clinic information system built using Flutter for the client-side and Laravel for the backend, with communication via REST API. The system implements a Model–View–Controller (MVC) architecture on the backend and uses MySQL as its database. The main features developed include the management of patient data, doctors, appointment schedules, queues, and medical records, all integrated into a single system. With the implementation of this architecture, it is hoped that clinic administration and service processes will become more effective and structured, making it easier for staff to access and manage information quickly and accurately.

B. Problem Statement

Based on the background described above, the clinical administrative processes that are still carried out manually give rise to various problems, including delays in patient data management, a high potential for recording errors, difficulties in managing appointment schedules and patient queues, and the ineffective storage of medical records. In addition, limited data integration forces staff to make duplicate entries, thereby reducing service efficiency.

The use of mobile applications connected to the backend via client–server architecture and REST APIs is one solution to address these issues. However, developing a system capable of integrating patient, doctor, appointment schedule, queue, and medical record modules into a single centralized platform still requires proper architectural design to ensure that data exchange occurs quickly, consistently, and is easy to maintain.

Based on these issues, this study focuses on the development of a mobile-based clinical information system using Flutter as the client application and Laravel as the backend, with REST API communication and the implementation of the Model–View–Controller (MVC) architecture, to produce a system that is integrated, easy to develop, and capable of supporting clinical administrative services more effectively.

C. Research Objectives

This study aims to develop the KlinikKu application, a mobile-based clinic information system that employs a client–server architecture using Flutter as the frontend and Laravel as the backend, which communicate via a REST API. Additionally, this study aims to implement the Model–View–Controller (MVC) architectural pattern in the backend to produce a more organized and easily maintainable application structure. The developed system is expected to integrate the management of patient data, doctors, appointment schedules, queues, and medical records, thereby making administrative processes and clinic services more effective, efficient, and structured.

D. Research Contributions

The main contribution of this research is the development of the KlinikKu application, a mobile-based clinic information system that integrates a Flutter application with a Laravel backend via a REST API within a client–server architecture. This research also applies the Model–View–Controller (MVC) pattern to the backend to improve the consistency of software development and utilizes Laravel Sanctum as an API authentication mechanism and Eloquent ORM for managing the MySQL database. In addition to producing a system that supports the integrated management of patient data, doctors, appointment schedules, queues, and medical records, this study is expected to serve as a reference for the application of Flutter and Laravel technologies in the development of mobile-based healthcare information systems.

II. LITERATURE REVIEW

A. Client–Server Architecture

Client–server architecture is one of the most widely used models in modern information system development. In this architecture, an application is divided into two main components: the client, which serves as the user interface, and the server, which acts as the service provider responsible for business logic processing, data management, and communication with the database. This separation of responsibilities allows for the development of systems that are more structured, easier to maintain, and more scalable.

In practice, client–server architecture generally employs a three-tier architecture consisting of the presentation layer, the application layer, and the data layer. The presentation layer provides the user interface; the application layer executes business processes and application services; and the data layer is responsible for data storage and management. The separation of these three layers allows each component to be developed or updated independently without affecting the entire system.

In this study, the three-tier architecture concept was implemented using Flutter as the presentation layer, Laravel REST API as the application layer, and MySQL as the data layer. Through this division, communication between the mobile app and the server can be carried out efficiently, thereby supporting the management of patient data, doctors, appointment schedules, queues, and medical records within a single integrated system.

B. Rest Api

The Representational State Transfer Application Programming Interface (REST API) is an architectural approach used to connect client applications to a server via the HTTP protocol. REST APIs utilize the concept of resources accessed using Uniform Resource Identifiers (URIs), while communication occurs through HTTP methods such as GET, POST, PUT, and DELETE.

The REST approach is stateless by nature; that is, every request sent by the client must include all necessary information, so the server does not store the status of previous communications. This characteristic offers several advantages, including ease of development, improved system scalability, and ease of integration with various platforms, including mobile applications.

In this study, the REST API is used as the communication channel between the Flutter application and the Laravel backend. All data management processes—such as user authentication, patient and doctor data management, appointment schedules, queues, and medical records—are handled through REST API endpoints that send and receive data in JavaScript Object Notation (JSON) format. This approach allows the mobile app and backend to be developed separately while remaining integrated through the same API service.

C. HTTP Protocol

Hypertext Transfer Protocol (HTTP) is a communication protocol used to exchange data between clients and servers in web- and mobile-based applications. In this communication mechanism, the client sends an HTTP request to the server; the server then processes the request and returns an HTTP response containing information about the processing results.

Every HTTP request consists of several components, such as the HTTP method, URI, headers, and message body, while an HTTP response contains a status code, headers, and a body that is typically sent in JSON format in REST API implementations. The use of status codes—such as 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, and 404 Not Found—helps clients determine the outcome of each request they send.

In this study, communication between the Flutter and Laravel applications is conducted using the HTTP protocol via a REST API. All data sent and received uses the JSON format, making the information exchange lightweight, fast, and easy to process by both platforms.

D. MVC Architecture

Model–View–Controller (MVC) is a software architecture pattern designed to separate the responsibilities of each application component, thereby simplifying the development, maintenance, and testing of the system. This architecture divides an application into three main components: Model, View, and Controller.

The Model is responsible for managing the application's data and business logic. This component interacts directly with the database to store, retrieve, update, and delete data. The View serves as the user interface that displays information to users based on data obtained from the Model. Meanwhile,

the Controller is responsible for receiving user requests, processing the application's logic, communicating with the Model, and then returning the results to the View.

The implementation of the MVC pattern offers several advantages, including improved application modularity, easier maintenance, reduced coupling between components, and a more structured development process. Therefore, the MVC pattern is widely used in modern web development frameworks, including Laravel, to produce applications that are easier to develop and have better code structure.

E. Laravel Framework

Laravel is a PHP-based framework that implements the Model–View–Controller (MVC) architecture to facilitate more structured web application development. This framework provides various features that support modern application development, such as a routing system, middleware, authentication, database migrations, the Eloquent ORM, and Resource Controllers for building REST APIs.

Through its routing system, Laravel is able to direct each HTTP request to the appropriate controller. The controller then processes the application's business logic and interacts with the model via the Eloquent ORM to perform operations on the database. The results of this processing are then returned to the client as a response, which in a REST API implementation typically uses the JSON format.

In this study, Laravel was used as the backend to manage all business processes for the KlinikKu application. This framework handles user authentication using Laravel Sanctum, data validation, REST API endpoint management, and Create, Read, Update, and Delete (CRUD) operations on patient, doctor, appointment schedule, queue, and medical record data stored in a MySQL database.

F. Flutter Framework

Flutter is an open-source mobile app development framework developed by Google. This framework uses the Dart programming language and employs the concept of cross-platform development, allowing a single codebase to be used to build apps on various operating systems, such as Android and iOS.

Flutter provides a variety of user interface components (widgets) that allow developers to build apps with a consistent look and feel and high performance. Additionally, the hot reload feature speeds up the development process because code changes are immediately reflected without having to rebuild the entire app.

In this study, Flutter serves as the client-side application used by administrators or clinic staff to access the system's services. All user interactions—such as logging in, managing patient and doctor data, appointment schedules, queues, and medical records—are carried out through communication with the Laravel backend using a REST API. Data received from the server in JSON format is then processed and displayed through the application interface, enabling users to access information quickly and in an integrated manner.

G. Related Works

Several previous studies have applied client–server architecture and REST APIs in the development of web- and mobile-based information systems. Research on the implementation of RESTful APIs using Laravel and Flutter shows that separating the client and server applications enhances development flexibility and facilitates cross-platform integration. This approach enables mobile applications to communicate efficiently with the backend through the exchange of data in JSON format.

Another study on web-based clinical medical record information systems shows that digitizing administrative processes can improve the effectiveness of managing patient data, physician information, appointment schedules, and medical records. The system has successfully replaced manual record-keeping, making information more structured and easily accessible.

Based on these studies, the development of KlinikKu combines the implementation of client–server architecture, REST APIs, the Flutter framework, and Laravel into a mobile-based clinic information system. Unlike previous studies, which generally focused on web-based applications or only discussed REST API communication, this study integrates the management of patients, doctors, appointment schedules, queues, and medical records into a single mobile application connected to the Laravel backend via REST API. Thus, this study is expected to provide an alternative implementation of a clinic information system that is more flexible, integrated, and easy to develop.

III. RESEARCH METHOD

A. Research Flow

This study employs a system development process that begins with problem identification and concludes with an evaluation of the implementation results. The first stage is problem identification, which involves analyzing the challenges encountered in clinical administrative processes—such as patient data management, appointment scheduling, queues, and medical records—that are still handled manually. Next, a system requirements analysis is conducted to determine the functional and non-functional requirements to be implemented in the application.

The next stage is system design, which includes designing the client–server architecture, database structure, and REST API, as well as implementing the Model–View–Controller (MVC) pattern in the Laravel backend and the Flutter application architecture on the client side. Once the design process is complete, the system is implemented using Flutter for the mobile app, Laravel for the REST API backend, and MySQL as the database.

The next stage is system testing using the Black Box Testing method to ensure that every feature operates in accordance with the functional requirements. The test results are then analyzed during the evaluation stage to determine whether the system has met the research objectives.

B. Requirement Analysis

1) Functional Requirements

Based on the results of the requirements analysis, the KlinikKu system has the following functional requirements.

- Users can authenticate themselves using the login and logout features.
- The system can display summary information on the dashboard page.
- The system can manage patient data, including adding, modifying, deleting, and viewing data.
- The system can manage doctor data.

- The system can manage examination schedules.
- The system can manage patient queue data.
- The system can manage medical record data.
- The system exchanges data between the Flutter and Laravel applications using a REST API.

2) Non-Functional Requirements

The system's non-functional requirements include the use of Flutter as the mobile app development framework, Laravel 13 as the backend framework, MySQL as the database management system, and an HTTP-based REST API with JSON data exchange format. Additionally, the system implements Laravel Sanctum as the API authentication mechanism and the Model–View–Controller (MVC) architectural pattern on the backend to improve software development consistency.

C. System Architecture

The system architecture used in this study employs a client–server model, in which the Flutter application acts as the client and Laravel acts as the server providing REST API services. All data communication is conducted using the HTTP protocol with JSON as the data exchange format.

On the client side, users interact through Flutter pages. Each page uses a service to communicate via HTTP with the Laravel REST API. The data received from the server is then mapped using a Dart model before being displayed on the application interface. Next, Laravel routes direct requests to the appropriate controller to execute the application's business logic. The controller then interacts with the model that manages access to the MySQL database. Once the process is complete, the Laravel controller returns the results in the form of a JSON response, which is sent back to the Flutter application via the service. The JSON data is then processed by the Flutter application before being displayed to the user. The data is processed by the Flutter controller before being displayed on the application page.

With this approach, frontend and backend development can be carried out separately without compromising integration between system components. This architecture also simplifies maintenance, facilitates the development of new features, and improves the application's scalability.

D. MVC Architecture

The application architecture in this study implements the Model–View–Controller (MVC) pattern in the Laravel backend, combined with the Flutter application architecture as the client. This approach aims to separate the responsibilities of each component so that the application development, maintenance, and testing processes become easier to carry out.

On the Laravel backend, the Model component is responsible for data management and interaction with the MySQL database via the Eloquent ORM. The models used include User, Patient, Doctor, Schedule, Queue, and MedicalRecord. All Create, Read, Update, and Delete (CRUD) operations are performed through these models.

The Controller component is tasked with receiving requests from clients, executing the application's business logic, validating data, and interacting with the models. The controllers used include AuthController, DashboardController, PatientController, DoctorController, ScheduleController, QueueController, and MedicalRecordController.

After the data is processed by the controller, Laravel sends the results to the client as a JSON response. This response contains the data required by the Flutter app to be displayed on the user's page. Using the JSON format ensures that data exchange between the Laravel backend and the Flutter app is simple and structured. All REST API endpoints are mapped to controllers via the routes/api.php file.

On the Flutter side, the application implements a structure consisting of Models, Services, Controllers, and Views (Pages). Models are used to represent objects received from JSON responses. Services are responsible for HTTP communication with the Laravel REST API. Controllers manage application logic and state changes, while Views display the user interface and receive user input. This structure enables the separation of concerns for each component, making the application more modular and easier to develop.

E. Database Design

The database used in this study is MySQL. The database was designed to support all business processes in the clinic's information system, ranging from user authentication to medical record management.

The database structure consists of several main entities: User, Patient, Doctor, Schedule, Queue, and MedicalRecord. Each entity is represented as a Laravel model, so interactions with the database are handled using the Eloquent ORM.

The relationships between entities are designed to support an integrated clinical service process. Patient data can include multiple medical records; doctors have practice schedules stored in the Schedule table; and patient queues link patient data to available service slots. All of this data is interconnected, allowing for consistent information management via a REST API.

The design of this database aims to minimize data redundancy, maintain data integrity, and facilitate the process of retrieving and updating data in each application module.

F. REST API Design

Communication between the Flutter application and the Laravel backend is carried out using a REST API with the HTTP protocol and the JSON data exchange format. Each feature in the application is implemented through endpoints connected to the Laravel controller using the routes/api.php file.

The authentication service consists of the POST /api/login and POST /api/logout endpoints to manage the user login and logout processes. After successfully authenticating, users can access the various services provided by the system.

The Dashboard module provides a GET endpoint at /api/dashboard to display a summary of system information. Furthermore, each main module implements CRUD operations via REST API endpoints. The Patient, Doctor, Schedule, Queue, and Medical Records modules each provide a GET endpoint to display data, a POST endpoint to add new data, a PUT endpoint to update existing data, and a DELETE endpoint to delete data.

A consistent endpoint design makes it easier for the Flutter application to access all backend services. Additionally, the use of HTTP methods in accordance with REST API standards makes cross-platform integration simpler, easier to

maintain, and easier to develop should new features be required in the future.

G. Technology Stack

The development of the KlinikKu app utilizes several integrated technologies. The Flutter framework is used to develop the Android-based mobile app, while Laravel 13 serves as the backend, providing REST API services. Communication between the frontend and backend occurs via the HTTP protocol using JSON for data exchange.

The system uses MySQL as its database to store all information regarding users, patients, doctors, schedules, queues, and medical records. On the backend, Laravel Sanctum is used as the API authentication mechanism, while the Eloquent ORM is utilized to facilitate interactions between the application and the database.

During the development process, REST API endpoints were tested using Thunder Client in Visual Studio Code to ensure that each endpoint functioned as intended, while Laragon was used as the local development environment to run Apache, PHP, and MySQL services. All of the application's source code was managed using Git so that the development process could be carried out collaboratively and well-documented.

IV. Results and Discussion

A. System Implementation Results

The KlinikKu app was successfully developed as a mobile-based clinic information system using a client-server architecture. The system consists of two main components: a Flutter app as the client and Laravel 13 as the backend API server. Flutter is responsible for providing the user interface and exchanging data via REST APIs, while Laravel handles the app's business processes, user authentication, data management, and communication with the MySQL database.

The system development implements a separation of responsibilities based on the Model-View-Controller (MVC) architecture in the Laravel backend. The Model component is used to represent data and manage interactions with the database using the Eloquent ORM; the Controller handles client requests and executes application logic; and the Laravel Controller is responsible for generating responses in JSON format, which are used for communication between the backend and the Flutter application.

On the Flutter side, the application uses a modular structure consisting of API configuration, Models, Pages, and Services. Pages serve as the user interface; Services are responsible for HTTP communication with the Laravel REST API; and Models are used to map JSON data to Dart objects. This structure makes the application development process more organized and easier to maintain.

.

B. Laravel 13 Backend Implementation

The KlinikKu app's backend was developed using the Laravel 13 framework, which serves as a REST API provider. The backend handles user authentication, clinic data management, and provides endpoints that can be used by the Flutter app.

The backend architecture follows the MVC pattern and consists of several key components, namely:

C. Model

Models are used to connect the application to database tables. The models used in the KlinikKu system consist of:

TABLE I. MODEL

Model	Function
User	Manages user data and access rights
Patient	Manage patient information
Doctor	Manage doctor information
Exam Schedule	Manage appointment schedules
Waiting List	Managing patient queue data
Medical Records	Managing patient examination histories

The model is responsible for managing relationships between tables using Laravel's Eloquent ORM.

D. Controller

The controller acts as a bridge between requests from the Flutter application and the database.

Controllers used:

TABEL II. DATABASE

Controller	Function
AuthController	Handles login, registration, and logout
DashboardController	Displays dashboard statistics
PasienController	CRUD operations on patient data
DokterController	CRUD operations on doctor data
JadwalController	CRUD for appointment schedules
AntrianController	Manages the queue process
RekamMedisController	Manages medical record data

The controller receives an HTTP request from the client, then executes the application logic before returning a JSON response.

V. Implementasi Authentication

The authentication system at KlinikKu uses Laravel Sanctum as its API security mechanism.

The authentication process involves several steps:

1. The user enters their email and password via the Flutter login page.
2. The data is sent using the HTTP POST method to the endpoint: POST /api/login
3. Laravel validates user data in the `users` table.
4. If the data is valid, the system generates an authentication token using Laravel Sanctum.
5. The token is sent back to the Flutter app.
6. The token is stored using SharedPreferences.

The implementation of Laravel Sanctum provides additional security because every request that requires authentication must include a valid access token.

VI. REST API IMPLEMENTATION

The REST API is used as the communication medium between Flutter (the client) and Laravel (the server).

The main endpoints used in the KlinikKu system are shown in Table III.

TABLE III. ENDPOINT REST API KLINIKKU

Module	Endpoint	Method
Register	/api/register	POST
Login	/api/login	POST
Logout	/api/logout	POST
Dashboard	/api/dashboard	GET
Patient	/api/pasien	GET, POST
Patient	/api/pasien/{id}	PUT, DELETE
Doctor	/api/dokter	GET, POST
Doctor	/api/dokter/{id}	PUT, DELETE
Schedule	/api/jadwal-pemeriksaan	GET, POST
Schedule	/api/jadwal-pemeriksaan/{id}	PUT, DELETE
Queue	/api/antrian	GET, POST
Queue	/api/antrian/{id}	PUT, DELETE
Medical Records	/api/rekam-medis	GET, POST
Medical Records	/api/rekam-medis/{id}	PUT, DELETE

A. Implementasi Database

The KlinikKu database uses MySQL with several main tables:

TABEL IV. DATABASE FUNCTIONS

Table	Function
users	Stores user data
patients	Stores patient data
doctors	Save doctor data
exam_schedule	Save examination schedule
queue	Save queue data
rekam_ medical_records	Save patient medical history

Flutter Client Implementation

The KlinikKu Flutter app uses a modular structure consisting of:

1) Config

The config folder is used to store API configurations.

`Example:`

- `api.dart`
- `api_config.dart`

This component is used to specify the API server address used by the application.

2) Models

Flutter models are used to convert JSON data from the REST API into Dart objects.

Models used:

TABEL V. DART

Model	Function
user.dart	User data
pasien.dart	Patient data
dokter.dart	Doctor data
jadwal_pemeriksaan.dart	Schedule data
antrian.dart	Queue Data
rekam_medis.dart	Medical records data

3) Pages

Pages serve as Views in a Flutter application.

Available pages:

TABEL V. PAGE

Page	Function
Login Page	User authentication
Register Page	User Registration
Home Page	App Dashboard
Patient Page	Patient Management
Doctor Page	Doctor Management
Schedule Page	Schedule Management
Page Queue	Waiting List Management
Medical Records Page	Medical Records Management

4) Services

The service is used to handle HTTP communication between Flutter and the Laravel API.

Example service:

- `auth_service.dart`
- `pasien_service.dart`
- `dokter_service.dart`
- `jadwal_service.dart`

- `antrian_service.dart`
- `rekam_medis_service.dart`

Each service is responsible for sending requests and receiving responses from a specific endpoint.

VII. Implementasi Fitur Sistem

A. Dashboard

The dashboard displays key clinic information, including:

- Total patients
- Total doctors
- Number of patients waiting in line
- Number of patients called
- Exams completed today
- Today's examination schedule

The dashboard helps users quickly access information about the clinic's status.

B. Patient Management

The patient feature provides the following functions:

- Displaying patient data
- Adding new patients
- Editing patient data
- Deleting patient data

Patient data managed:

- Name
- Date of Birth
- Gender
- Address
- Phone number

C. Doctor Management

The Doctor module is used to manage information about medical staff.

Doctor's information:

- Doctor's Name
- Specialty
- Phone number

Features:

- Add a doctor
- Edit doctor
- Delete doctor
- View doctor list

D. Appointment Scheduling

The schedule module is used to manage patient appointments.

Stored data:

- Patients
- Doctors
- Examination date
- Examination time
- Patient's complaints

E. Queue Management

The queue system uses ticket numbers such as:

- A001
- A002
- A003

Queue status:

- Waiting
- Called
- Completed

This feature helps clinic staff manage the patient care process.

F. Medical Records

The medical records module stores patients' medical histories.

Data managed:

- Patients
- Doctors
- Examination dates
- Diagnosis
- Prescriptions
- Notes

VIII. System Testing

Testing was performed using Thunder Client to ensure that all REST API endpoints function as required.

The testing method used was Black Box Testing, focusing on system inputs and outputs.

TABLE VI. API Test Results

Test	Endpoint	Results
User Registration	POST /api/register	Success
User login	POST /api/login	Success
Retrieve patient data	GET /api/pasien	Success
Add patient	POST /api/pasien	Success
Update patient	PUT /api/pasien/{id}	Success
Delete patient	DELETE /api/pasien/{id}	Success
Retrieve doctor data	GET /api/dokter	Success
Managing the queue	API Antrian	Success
Managing Medical Records	API Rekam Medis	Success

References

[1] U. Manu, R. Noviana, and J. M. R. No, "Analisis kualitas aplikasi Unit Link menggunakan metode ISO 25010 (studi kasus PT Asuransi Jiwasraya Persero)," *Jurnal Ilmiah MATRIK*, vol. 24, no. 2, 2022.

[2] Z. Subecz, "Web-development with Laravel framework," *Gradus*, vol. 8, no. 1, pp. 211–218, 2021.

[3] J. Jasri, "Arsitektur three tier (client server programing) pada aplikasi perpustakaan Fakultas Teknik Universitas Muhammadiyah Bengkulu," *Journal of Technopreneurship and Information System*, vol. 1, no. 1, pp. 19–25, 2018.

[4] A. Suryadi, Y. W. T. Arif, and N. S. Novitasari, "Rancang bangun sistem informasi rekam medis klinik rawat jalan berbasis web,"

Infokes: Jurnal Ilmiah Rekam Medis dan Informatika Kesehatan, vol. 12, no. 1, pp. 37–43, 2022.

[5] D. P. Pop and A. Altar, "Designing an MVC model for rapid web application development," *Procedia Engineering*, vol. 69, pp. 1172–1179, 2014.

[6] V. Surwase, "REST API modeling languages—A developer's perspective," *International Journal of Science Technology & Engineering*, vol. 2, no. 10, pp. 634–637, 2016.

[7] A. Sunardi and Suharjito, "MVC architecture: A comparative study between Laravel framework and Slim framework in freelancer project monitoring system web based," *Procedia Computer Science*, vol. 157, pp. 134–141, 2019.

[8] R. Sridaran, G. Padmavathi, K. Iyakutti, and M. N. S. Mani, "SPIM architecture for MVC based web applications," *Journal of Advanced Networking and Applications*, vol. 1, no. 1, pp. 63–68, 2010.

[9] J. Deacon, *Model–view–controller (MVC) architecture*. 2009. [Online]. Available: http://www.jdl.co.uk/briefings/MVC.pdf

[11] S. Kosasi, "Perancangan aplikasi point of sale dengan arsitektur client/server berbasis Linux dan Windows," *Creative Information Technology Journal*, vol. 1, no. 2, pp. 116–127, 2014.

[12] R. Fielding, M. Nottingham, and J. Reschke, Eds., *RFC 9110: HTTP Semantics*, RFC 9110, Internet Engineering Task Force (IETF), Jun. 2022.